

gasmodel: An R Package for Generalized Autoregressive Score Models

Vladimír Holý

Prague University of Economics and Business
Winston Churchill Square 1938/4, 130 67 Prague 3, Czechia
vladimir.holy@vse.cz

Abstract: Generalized Autoregressive Score (GAS) models are a class of observation-driven time series models that employ the score to dynamically update time-varying parameters of the underlying probability distribution. GAS models have been extensively studied and numerous variants have been proposed in the literature to accommodate diverse data types and probability distributions. This paper introduces the **gasmodel** package, which has been designed to facilitate the estimation, forecasting, and simulation of a wide range of GAS models. The package provides a rich selection of distributions, offers flexible options for specifying dynamics, and allows to incorporate exogenous variables. Model estimation utilizes the maximum likelihood method and the Hessian matrix.

Keywords: Generalized Autoregressive Score Models, Dynamic Conditional Score Models, Score-Driven Models, R.

JEL Classification: C22, C87.

1 Introduction

The generalized autoregressive score (GAS) models, introduced by [Creal *et al.* \(2013\)](#) and [Harvey \(2013\)](#), have emerged as a valuable and contemporary framework for time series modeling. These models, also referred to as dynamic conditional score (DCS) models or score-driven models, offer flexibility by accommodating various underlying probability distributions and time-varying parameters. GAS models are observation-driven, effectively capturing the dynamic behavior of time-varying parameters through the autoregressive term and the score, i.e., the gradient of the log-likelihood function. Within the GAS framework, it is possible to formulate a wide range of dynamic models for any type of data.

There are several packages and code available in R that handle GAS models. One notable package is **GAS** developed by [Ardia *et al.* \(2019\)](#), which provides functionality for both univariate and multivariate GAS models. The current version, 0.3.4, supports 16 distributions. However, the model specification in the **GAS** package is somewhat limited, only allowing for basic dynamics without the inclusion of exogenous variables. Additionally, this package lacks distributions for certain more specialized data types, such as circular, compositional, and ranking data. The package thus supports only a limited selection of GAS models found in the literature¹. Another relevant R package is **betategarch** by [Sucarrat \(2013\)](#), which deals specifically with the Beta-Skew-t-EGARCH model, a GAS model for time-varying volatility based on the Student's t-distribution. In Python, the **PyFlux** library by [Taylor \(2018\)](#) deals with time series analysis and features various GAS models including the Beta-Skew-t-EGARCH model, standard GAS models, GAS random walk models, GAS pairwise comparison models, and GAS regression models. In Julia, the **ScoreDrivenModels.jl** package by [Bodin *et al.* \(2020\)](#) provides a framework for standard GAS models. The **Time Series Lab** program by [Lit *et al.* \(2021\)](#) is a stand-alone GUI application designed to model and forecast time series, including standard GAS models, GAS pairwise comparison models, and GAS regression models. Additional R, MATLAB, and Ox code for some specific GAS models, often associated with individual research papers, can be found on the www.gasmodel.com website.

In this paper, we present the **gasmodel** package, which is designed to provide comprehensive functionality that encompasses a wide range of GAS models documented in the existing literature. It

¹For a more detailed comparison of the **gasmodel** and **GAS** packages, see Appendix A.

Table 1: List of available distributions and their parametrizations. First parametrization is the default.

Label	Distribution	Dimension	Data Type	Parametrizations
alaplace	Asymmetric Laplace	Univariate	Real	meanscale
bernoulli	Bernoulli	Univariate	Binary	prob
beta	Beta	Univariate	Interval	conc, meansize, meanvar
bisa	Birnbaum–Saunders	Univariate	Duration	scale
cat	Categorical	Multivariate	Categorical	worth
dirichlet	Dirichlet	Multivariate	Compositional	conc
dpois	Double Poisson	Univariate	Count	mean
exp	Exponential	Univariate	Duration	scale, rate
gamma	Gamma	Univariate	Duration	scale, rate
gengamma	Generalized Gamma	Univariate	Duration	scale, rate
geom	Geometric	Univariate	Count	mean, prob
laplace	Laplace	Univariate	Real	meanscale
mvnorm	Multivariate Normal	Multivariate	Real	meanvar
mvt	Multivariate Student’s t	Multivariate	Real	meanvar
negbin	Negative Binomial	Univariate	Count	nb2, prob
norm	Normal	Univariate	Real	meanvar
pluce	Plackett–Luce	Multivariate	Ranking	worth
pois	Poisson	Univariate	Count	mean
skellam	Skellam	Univariate	Integer	meanvar, diff, meandisp
t	Student’s t	Univariate	Real	meanvar
vonmises	von Mises	Univariate	Circular	meanconc
weibull	Weibull	Univariate	Duration	scale, rate
zigeom	Zero-Inflated Geometric	Univariate	Count	mean
zinegbin	Zero-Inflated Negative Binomial	Univariate	Count	nb2
zipois	Zero-Inflated Poisson	Univariate	Count	mean
ziskellam	Zero-Inflated Skellam	Univariate	Integer	meanvar, diff, meandisp

offers versatile model specification and core features available for the entire spectrum of implemented distributions. The current version of the package, 0.5.1, offers a selection of 26 distributions, catering to various univariate and multivariate data types such as binary, categorical, ranking, count, integer, circular, interval, compositional, duration, and real data. A comprehensive list of these distributions is provided in Table 1. Model specification within the package allows for flexible customization, enabling users to incorporate different parametrizations, exogenous variables, joint and separate modeling of exogenous variables and dynamics, higher score and autoregressive orders, custom and unconditional initial values of time-varying parameters, fixed and bounded values of coefficients, and missing values. Model estimation is performed by the maximum likelihood method and the Hessian matrix. Furthermore, the package offers a range of functionalities including forecasting, simulation, bootstrapping, and assessment of parameter uncertainty. Comprehensive documentation is provided with the package, offering details on each distribution and its corresponding parametrizations.

The **gasmodel** package is accessible on CRAN at cran.r-project.org/package=gasmodel. Additionally, users can find the development version of the package on GitHub at github.com/vladimirholy/gasmodel, providing them with the opportunity to report any bugs or issues they encounter.

The rest of the paper is as follows. In Section 2, we outline the key characteristics of GAS models. In Section 3, we present an overview of the **gasmodel** package. In Section 4, we present two case studies demonstrating the practical application of the package. In Section 5, we discuss limitations and customization. We conclude the paper in Section 6.

2 Generalized Autoregressive Score Models

2.1 Background

The concept of utilizing the score as a driving mechanism for dynamics in time series was independently developed at both Vrije Universiteit Amsterdam and the University of Cambridge. At

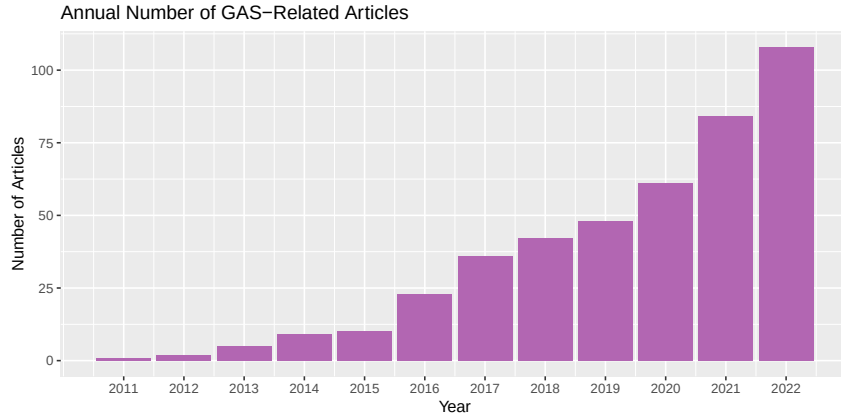


Figure 1: The annual number of articles containing phrase “generalized autoregressive score” or “dynamic conditional score” from 2011 to 2022 according to Scopus.

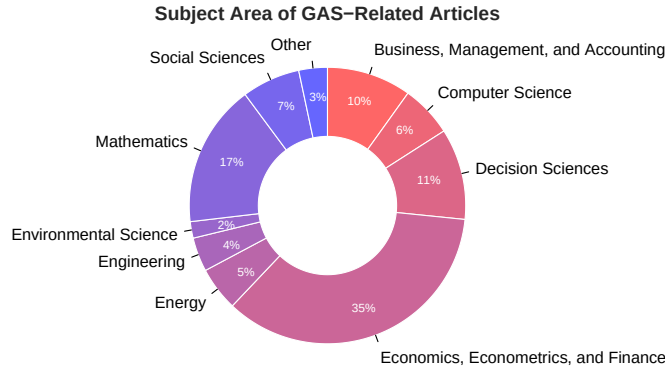


Figure 2: The subject area of articles containing phrase “generalized autoregressive score” or “dynamic conditional score” from 2011 to 2022 according to Scopus.

Vrije Universiteit Amsterdam, researchers established a comprehensive general methodology that encompasses various models driven by the score, known as the generalized autoregressive score (GAS) models. The initial findings were presented in a working paper [Creal et al. \(2008\)](#), which was subsequently published as [Creal et al. \(2013\)](#). At the University of Cambridge, the initial focus was on a specific model that employed the Student’s t-distribution with dynamic volatility, named Beta-t-(E)GARCH. This approach was introduced in a working paper [Harvey and Chakravarty \(2008\)](#). The book by [Harvey \(2013\)](#) explores a variety of dynamic location and scale models driven by the score, referring to them as dynamic conditional score (DCS) models. Both [Creal et al. \(2013\)](#) and [Harvey \(2013\)](#) are widely recognized as seminal contributions to the literature on GAS models. More recently, in order to reconcile different terminologies used in the literature, the term “score-driven models” has also emerged as a synonymous label.

Figures 1 and 2 illustrate the continuous growth of the GAS literature, encompassing a wide range of subject areas. The Scopus database reports 429 articles containing phrase “generalized autoregressive score” or “dynamic conditional score”, as of December 31, 2022. The website www.gasmodel.com lists 288 articles, working papers, and books on GAS models, as of September 28, 2022.

2.2 Basic Notation

The goal is to model time series y_t , $t = 1, \dots, T$, which can be univariate or multivariate, continuous or discrete. Let f_t denote the vector of time-varying parameters and g the vector of static parameters. Let $p(y_t|f_t, g)$ denote the density function in the case of a continuous variable, or the probability mass function in the case of a discrete variable.

Constructing a model involves two main components: selecting an appropriate distribution and specifying the dynamics of its time-varying parameters.

2.3 Score as the Key Ingredient

In GAS models, the key ingredient driving the dynamics of f_t is the score, i.e., the gradient of the log-likelihood function,

$$\nabla(y_t, f_t) = \frac{\partial \ln p(y_t|f_t, g)}{\partial f_t}. \quad (1)$$

The score has zero expected value and its variance is known as the Fisher information,

$$\mathcal{I}(f_t) = \mathbb{E} \left[\left(\frac{\partial \ln p(y_t|f_t, g)}{\partial f_t} \right)^2 \middle| f_t, g \right]. \quad (2)$$

The score quantifies the discrepancy between the fitted distribution, determined by the parameter f_t , and a particular observation y_t . As such, it can be employed as a correction term following the realization of an observation. When the score is positive, it suggests that the parameter of interest should be increased to better accommodate the observed data. Conversely, when the score is negative, decreasing the parameter would help in aligning the distribution with the observation. When the score is zero, it indicates that the current parameter value represents the optimal fit for the specific observation at hand.

An advantage of the score is that it takes into account the shape of the distribution. To illustrate this point, [Creal *et al.* \(2013\)](#) consider two GARCH models: one based on the normal distribution and another based on the Student's t-distribution. Now, imagine an extreme observation occurs. Due to its heavier tails, the Student's t-distribution assigns a higher probability to such extreme observations compared to the normal distribution. Crucially, this distinction is also mirrored in the score. Specifically, when assuming the normal distribution, the score for the extreme observation will have a significantly higher absolute value compared to when assuming the Student's t-distribution. The dynamics can thus reflect the shape of the distribution.

The simple difference between expectation and realization, commonly used as a correction term in various time series models, may not always be effective for distributions with specific support. [Harvey *et al.* \(2019\)](#) highlight this limitation in the context of circular time series. To illustrate this, let us suppose the expected value of an observation is 0.01, but the actual observation turns out to be 6.27. Although the numerical difference between these values is substantial, their corresponding angles are very similar as values 0 and 2π represent the exact same angle. This discrepancy highlights the inadequacy of using a simple difference metric. On the other hand, the score respects the circular nature of the data. For instance, when working with the von Mises distribution characterized by a time-varying location parameter μ_t and a static concentration parameter ν , the score for μ_t is equal to $\nu \sin(y_t - \mu_t)$. By employing the sine function, the score accounts for the circularity of the data and ensures that the angular differences are appropriately considered during the analysis.

2.4 Dynamics of Time Varying Parameters

In GAS models, time-varying parameters f_t follow the recursion

$$f_t = \omega + \sum_{j=1}^P \alpha_j S(f_{t-j}) \nabla(y_{t-j}, f_{t-j}) + \sum_{k=1}^Q \varphi_k f_{t-k}, \quad (3)$$

where ω is a vector of constants, α_j are score parameters, φ_k are autoregressive parameters, and $S(f_t)$ is a scaling function for the score. In the majority of empirical studies, it is common practice to set the score order P and the autoregressive order Q to 1. Furthermore, one of three scaling functions is typically chosen: the unit function, the inverse of the Fisher information, or the square root of the inverse of the Fisher information. When the latter is used, the scaled score has unit variance. However, the choice of the scaling function is not always a straightforward task and is closely tied to the underlying distribution. For a detailed discussion on this matter, see [Holý \(2020\)](#).

The dynamics of the model can be expanded to incorporate exogenous variables as

$$f_t = \omega + \sum_{i=1}^M \beta_i x_{ti} + \sum_{j=1}^P \alpha_j S(f_{t-j}) \nabla(y_{t-j}, f_{t-j}) + \sum_{k=1}^Q \varphi_k f_{t-k}, \quad (4)$$

where β_i are regression parameters associated with the exogenous variables x_{ti} . Alternatively, a different model can be obtained by defining the recursion in the fashion of regression models with dynamic errors as

$$f_t = \omega + \sum_{i=1}^M \beta_i x_{ti} + e_t, \quad e_t = \sum_{j=1}^P \alpha_j S(f_{t-j}) \nabla(y_{t-j}, f_{t-j}) + \sum_{k=1}^Q \varphi_k e_{t-k}. \quad (5)$$

The key distinction between the two models lies in the impact of exogenous variables on f_t . Specifically, in the latter model formulation, the exogenous variables influence solely the concurrent parameter f_t , while in the former model, they additionally affect all future parameters through the autoregressive term. In a stationary model without exogenous variables, the two specifications are equivalent, although with differently parametrized intercept. When numerically finding the values of the parameters, the latter model can converge faster as ω is “disconnected” from φ_k .

Other model specifications can be obtained by imposing various restrictions on ω , β_i , α_j , or φ_k . In addition, it is possible to have different orders P and Q for individual parameters when multiple parameters are time-varying. Furthermore, the set of exogenous variables can also vary for different parameters.

The recursive nature of f_t necessitates the initialization of the first few elements $f_1, \dots, f_{\max\{P, Q\}}$. A sensible approach is to set them to the long-term value (in the case of a stationary model omitting exogenous variables),

$$\bar{f} = \begin{cases} \frac{1}{1 - \sum_{k=1}^Q \varphi_k} & \text{in model (4),} \\ \omega & \text{in model (5).} \end{cases} \quad (6)$$

Alternatively, if additional information is available, the initial elements can be set to a specified value.

2.5 Maximum Likelihood Estimation

GAS models can be straightforwardly estimated by the maximum likelihood method. Let $\theta = (\omega, \beta_1, \dots, \beta_M, \alpha_1, \dots, \alpha_P, \varphi_1, \dots, \varphi_Q, g)'$ denote the vector of all parameters to be estimated. The estimate $\hat{\theta}$ is then obtained by maximizing the full log-likelihood as

$$\hat{\theta} \in \arg \max_{\theta} \sum_{t=1}^T \ln p(y_t | f_t, g). \quad (7)$$

Alternatively, the conditional log-likelihood can be maximized, which excludes the initial $\max\{P, Q\}$ terms. The maximization of the log-likelihood function can be accomplished using various general-purpose algorithms designed for solving nonlinear optimization problems.

The standard errors of the estimated parameters can be obtained using the standard maximum likelihood asymptotics. Under appropriate regularity conditions, the maximum likelihood estimator $\hat{\theta}$ is consistent and asymptotically normal. Specifically, it satisfies:

$$\sqrt{T}(\hat{\theta} - \theta_0) \xrightarrow{d} N(0, -H^{-1}), \quad (8)$$

where θ_0 represents the true parameter values and H denotes the asymptotic Hessian of the log-likelihood, defined as

$$H = \text{plim}_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \frac{\partial^2 \ln p(y_t | f_t, g)}{\partial \theta_0 \partial \theta_0'}. \quad (9)$$

In finite samples, the asymptotic Hessian H can be approximated by the empirical Hessian of the log-likelihood evaluated at the estimated parameter values $\hat{\theta}$. This empirical Hessian provides an estimate of the curvature of the log-likelihood function and serves as a practical substitute for the true asymptotic Hessian when finite-sample inference is required.

The conditions for the consistency and asymptotic normality of the estimator depend on the specific distributional assumptions and dynamics of the model and need to be verified on a case-by-case basis. Each distribution may have its own specific characteristics and requirements for maximum likelihood estimation. For the general asymptotic theory regarding GAS models and maximum likelihood estimation, see Blasques *et al.* (2014), Blasques *et al.* (2018), and Blasques *et al.* (2022b).

2.6 Theoretical and Empirical Properties

The use of the score for updating time-varying parameters is optimal in an information theoretic sense. For an investigation of the optimality properties of GAS models, see Blasques *et al.* (2015) and Blasques *et al.* (2021).

Generally, the GAS models perform quite well when compared to alternatives, including parameter-driven models. For a comparison of the GAS models to alternative models, see Koopman *et al.* (2016) and Blazsek and Licht (2020).

2.7 Notable Models

The GAS class includes many well-known econometric models, such as the generalized autoregressive conditional heteroskedasticity (GARCH) model of Bollerslev (1986) based on the normal distribution, the autoregressive conditional duration (ACD) model of Engle and Russell (1998) based on the exponential distribution, and the count model of Davis *et al.* (2003) based on the Poisson distribution.

More recently, a variety of novel score-driven models has been proposed, such as the Beta-t-(E)GARCH model of Harvey and Chakravarty (2008), a multivariate Student's t volatility model of Creal *et al.* (2011), a Dirichlet model of Calvori *et al.* (2013), the GRAS copula model of De Lira Salvatierra and Patton (2015), the realized Wishart-GARCH model of Hansen *et al.* (2016), a bimodal Birnbaum–Saunders model of Fonseca and Cribari-Neto (2018), a Skellam model of Koopman *et al.* (2018), a circular model of Harvey *et al.* (2019), a Bradley–Terry model of Gorgi *et al.* (2019), a bivariate Poisson model of Koopman and Lit (2019), a censoring model of Harvey and Ito (2020), a zero-inflated negative binomial model of Blasques *et al.* (2022a), a double Poisson mixture model of Holý and Tomanová (2022), a ranking model of Holý and Zouhar (2022), and a Tobit model of Harvey and Liao (2023).

For an overview of various GAS models, see Artemova *et al.* (2022) and Harvey (2022).

3 Features of the Package

3.1 Model Specification and Estimation

The heart of the **gasmodel** package is the `gas()` function, which serves as a powerful tool for estimating both univariate and multivariate GAS models. This function offers extensive flexibility with its wide range of arguments:

```
R> gas(y, x = NULL, distr, param = NULL, scaling = "unit", regress = "joint",
+     p = 1L, q = 1L, par_static = NULL, par_link = NULL, par_init = NULL,
+     lik_skip = 0L, coef_fix_value = NULL, coef_fix_other = NULL,
+     coef_fix_special = NULL, = NULL, coef_bound_upper = NULL, coef_start = NULL,
+     optim_function = wrapper_optim_nloptr, optim_arguments = list(opts =
```



```

+ list(algorithm = "NLOPT_LN_NELDERMEAD", xtol_rel = 0, maxeval = 1e+06)),
+ hessian_function = wrapper_hessian_stats, hessian_arguments = list(),
+ print_progress = FALSE)

```

However, at its core, it only requires two essential inputs: a time series **y** and a distribution **distr**. All other arguments come with default values, ensuring that the function can be readily used even with minimal specifications.

A time series **y** can be represented as either a vector of length T or a $T \times 1$ matrix in the case of univariate series. In the multivariate case, it should be a $T \times N$ matrix, where N denotes the dimension of the series.

Additionally, there is an option to include exogenous variables **x**. When incorporating a single variable that is common for all time-varying parameters, a numeric vector of length T can be provided. For multiple variables that are common for all time-varying parameters, a $T \times M$ numeric matrix can be used. In cases where there are individual variables for each time-varying parameter, a list of numeric vectors or matrices following the aforementioned formats can be utilized. To control whether the variables are included in the dynamics equation together, as in (4), the argument **regress** can be set to "joint". Alternatively, if separate equations for dynamics and regression are preferred, as in (5), **regress** can be set to "sep".

The selection of the distribution in the **gas()** function is determined by the **distr** argument. Some distributions have multiple parametrizations available, which can be specified using the **param** argument. It is important to note that certain parameters may have restrictions imposed on them, and these restrictions should be considered in the model dynamics. However, it may not always be possible to satisfy these restrictions, or it may require additional constraints on the coefficients controlling the dynamics. To handle parameter restrictions, it is generally recommended to use a link function that transforms the parameters into unrestricted real numbers. By default, the logarithmic function is applied to time-varying parameters in the interval $(0, \infty)$, while the logistic function is used for time-varying parameters in the interval $(0, 1)$. The static parameters are unaffected. This behavior can be modified by the **par_link** argument, which takes the form of a logical vector. The TRUE values indicate that the logarithmic/logistic link is applied to the corresponding parameters. The list of available distributions and their parametrizations can be obtained using the **distr()** function. Alternatively, Table 1 provides the relevant information.

The determination of time-varying and static parameters is guided by the **par_static** argument, which takes the form of a logical vector. The TRUE values indicate static parameters. By default, the first parameter of the distribution is considered time-varying, while the remaining parameters are treated as static. The score order P and the autoregressive order Q are selected by the **p** and **q** arguments respectively. These arguments can take either a single non-negative integer or a vector of non-negative integers when different orders are required for different parameters.

The choice of scaling function for the score is determined by the **scaling** argument. The supported scaling options include the unit scaling (**scaling** = "unit"), the scaling based on the inverse of the Fisher information matrix (**scaling** = "fisher_inv"), and the scaling based on the inverse square root of the Fisher information matrix (**scaling** = "fisher_inv_sqrt"). The latter two scalings utilize the Fisher information for the time-varying parameters exclusively. If the preference is to use the full Fisher information matrix, which includes both time-varying and static parameters, the "full_fisher_inv" or "full_fisher_inv_sqrt" scaling options can be selected. For the individual Fisher information associated with each parameter, the "diag_fisher_inv" and "diag_fisher_inv_sqrt" scaling options are available. It should be noted that when the parametrization is orthogonal (see **distr()**), there are no differences among these scaling variants.

The first $\max\{P, Q\}$ initial values of the time-varying parameters are by default set to their long-term values (6). It is also possible to assign specific values to the initial parameters using the **par_init** argument. During the maximization of the log-likelihood, the initial values can be included, resulting in the computation of the full likelihood, which is the default option. Alternatively, the initial values can be omitted, leading to the computation of the conditional likelihood by specifying **lik_skip** = NULL. To exclude a specified number of first few values from the likelihood calculation, a non-negative integer can be provided to **lik_skip**.

Restrictions on estimated coefficients can be enforced using several arguments. The `coef_fix_value` argument allows coefficients to be fixed at specific values, using a numeric vector where `NA` values indicate coefficients that are not fixed. To set coefficients as linear combinations of other coefficients, the `coef_fix_other` argument can be used. It requires a square matrix with multiples of the estimated coefficients, which are added to the fixed coefficients. A coefficient given by row is fixed on coefficient given by column. By this logic, all rows corresponding to the estimated coefficients should contain only `NA` values. All columns corresponding to the fixed coefficients should also contain only `NA` values. For convenience, common coefficient structures can be specified by name using the `coef_fix_special` argument. Examples include `panel_structure`, `zero_sum_intercept`, and `random_walk`. Section 4.2 provides demonstrations of their usage. To impose lower and upper bounds on coefficients, the `coef_bound_lower` and `coef_bound_upper` arguments can be utilized, respectively.

The `coef_start` argument allows for the specification of the starting values of coefficients used in the optimization process. If no values are provided, the starting values are automatically selected from a small grid of values. To define the optimization function, the `optim_function` argument is used. The function should be formatted according to the required specifications. Two wrapper functions are available for convenience: `wrapper_optim_stats()`, which utilizes the `optim()` function from the `stats` package, and `wrapper_optim_nloptr()`, which utilizes the `nloptr()` function from the `nloptr` package. Additional arguments can be passed to the optimization function as a list using the `optim_arguments` argument. Similarly, the Hessian matrix can be computed using the function specified in the `hessian_function` argument. Three wrappers are available: `wrapper_hessian_stats` for the `optimHess()` function from the `stats` package, `wrapper_hessian_pracma` for the `hessian()` function from the `pracma` package, and `wrapper_hessian_numderiv` for the `hessian()` function from the `numDeriv` package. Additional arguments for the Hessian function can be passed as a list using the `hessian_arguments` argument. If desired, a detailed computation report can be continuously printed by setting the `print_progress` argument to `TRUE`.

The function returns a list of S3 class `gas`. This list consists of five components: `data`, `model`, `control`, `solution`, and `fit`, each of which is also a list. The `data` component contains the supplied time series and exogenous variables. The `model` component contains the specification of the model structure and size. The `control` component contains the settings that control the optimization and Hessian computation. The `solution` component contains the raw results of the optimization and Hessian computation. Lastly, and most importantly, the `fit` component contains comprehensive estimation results. When an object of the `gas` class is printed, it provides a concise summary similar to the `summary.lm()` function from the `stats` package (refer to Sections 4.1 and 4.2 for examples). Various generic functions can be applied to `gas` objects, including `summary()`, `plot()`, `coef()`, `vcov()`, `fitted()`, `residuals()`, `logLik()`, `AIC()`, `BIC()`, and `confint()`.

3.2 Forecasting

Forecasting of GAS models is performed using the `gas_forecast()` function. This function offers two forecasting methods. The `mean_path` method filters the time-varying parameters based on zero score and then generates the mean of the time series. The `simulated_paths` method repeatedly simulates time series, simultaneously filters time-varying parameters, and then estimates mean, standard deviation, and quantiles. See Blasques *et al.* (2016b) for more details on this method.

To use the `gas_forecast()` function, an estimated GAS model is required. Typically, the output of the `gas()` function (a `gas` object) can be supplied via the `gas_object` argument:

```
R> gas_forecast(gas_object, method = "mean_path", t_ahead = 1L, x_ahead = NULL,
+   rep_ahead = 1000L, quant = c(0.025, 0.975))
```

Alternatively, multiple arguments including the data, model specification, and estimated coefficients can be manually specified:

```
R> gas_forecast(method = "mean_path", t_ahead = 1L, x_ahead = NULL,
+   rep_ahead = 1000L, quant = c(0.025, 0.975), y, x = NULL, distr, param = NULL,
```



```
+   scaling = "unit", regress = "joint", p = 1L, q = 1L, par_static = NULL,
+   par_link = NULL, par_init = NULL, coef_est = NULL)
```

The forecasting method is determined by the `method` argument. The number of observations to forecast can be specified using the `t_ahead` argument. If exogenous variables are utilized, their values must be provided for the forecasted period using the `x_ahead` argument. For the `simulated_paths` method, the number of simulations can be controlled using the `rep_ahead` argument, and the desired quantiles can be specified using the `quant` argument.

The function returns a list of S3 class `gas_forecast` with three components: `data`, `model`, `forecast`. The `data` component contains the supplied time series and exogenous variables. The `model` component contains the specification of the model structure and size. The `forecast` component contains the mean of the forecasted observations, along with standard deviations and quantiles if the `simulated_paths` method is used. Available generic functions are `summary()` and `plot()`.

3.3 Simulation

Basic simulation of GAS models is handled by the `gas_simulate()` function.

The `gas_simulate()` function requires supplying the coefficients using the `coef_est` argument and specifying the model using arguments `distr`, `param`, `scaling`, `regress`, `p`, `q`, `par_static`, `par_link`, `par_init`, and, in the case of multivariate models, the dimension `n`:

```
R> gas_simulate(t_sim = 1L, x_sim = NULL, distr, param = NULL, scaling = "unit",
+   regress = "joint", n = NULL, p = 1L, q = 1L, par_static = NULL,
+   par_link = NULL, par_init = NULL, coef_est = NULL)
```

Alternatively, only a `gas` object containing a model estimated by the `gas()` function can be provided using the `gas_object` argument:

```
R> gas_simulate(gas_object, t_sim = 1L, x_sim = NULL)
```

The number of observations to simulate can be specified using the `t_sim` argument. If exogenous variables are utilized, their values must be provided for the simulation sample using the `x_sim` argument.

The function returns a list of S3 class `gas_simulate` with three components: `data`, `model`, `simulation`. The `data` component contains the exogenous variables, if supplied. The `model` component contains the specification of the model structure and size. The `simulation` component contains the simulated time series, time-varying parameters, and scores. Available generic functions are `summary()` and `plot()`.

3.4 Bootstrapping

To compute standard deviations and confidence intervals of the estimated coefficients in GAS models, the package provides the `gas_bootstrap()` function. This function employs the bootstrapping technique to estimate the uncertainty associated with the coefficients. The `parametric` method involves repeatedly simulating time series using the parametric model and re-estimating the coefficients based on the simulated data. The `simple_block`, `moving_block`, and `stationary_block` methods execute the circular block bootstrap with fixed non-overlapping blocks, fixed overlapping blocks, and randomly sized overlapping blocks, respectively.

The `gas_bootstrap()` function requires an estimated GAS model with optimization settings as inputs. The best way is to simply supply a `gas` object to the `gas_object` argument:

```
R> gas_bootstrap(gas_object, method = "parametric", rep_boot = 1000L,
+   block_length = NULL, quant = c(0.025, 0.975), parallel_function = NULL,
+   parallel_arguments = list())
```

Alternatively, the individual arguments including the data, model specification, estimated coefficients, and optimization setting can be provided:

```
R> gas_bootstrap(method = "parametric", rep_boot = 1000L, block_length = NULL,
+   quant = c(0.025, 0.975), y, x = NULL, distr, param = NULL, scaling = "unit",
+   regress = "joint", p = 1L, q = 1L, par_static = NULL, par_link = NULL,
+   par_init = NULL, lik_skip = 0L, coef_fix_value = NULL, coef_fix_other = NULL,
+   coef_fix_special = NULL, coef_bound_lower = NULL, coef_bound_upper = NULL,
+   coef_est = NULL, optim_function = wrapper_optim_nloptr, optim_arguments = list(opts =
+   list(algorithm = "NLOPT_LN_NELDERMEAD", xtol_rel = 0, maxeval = 1e+06)),
+   parallel_function = NULL, parallel_arguments = list())
```

The bootstrapping method is determined by the `method` argument. The number of bootstrap samples is specified by the `rep_boot` argument. For the `simple_block` and `moving_block` methods, the fixed size of blocks must be specified by the `block_length` argument. For the `stationary_block` method, the mean size of blocks must be specified by the `block_length` argument. The desired quantiles can be specified using the `quant` argument. As bootstrapping can be computationally very demanding, parallelization is achievable by employing the `parallel_function` argument, which expects a function similar to `lapply()`, allowing the application of a function over a list. Two wrapper functions are available for convenience: `wrapper_parallel_multicore()`, which utilizes the multi-core parallelization functionality from the `parallel` package, and `wrapper_parallel_snow()`, which utilizes the snow parallelization functionality from the `parallel` package. Additional arguments can be passed to the parallelization function as a list using the `parallel_arguments` argument. If `parallel_function` is set to `NULL`, no parallelization is employed and `lapply()` is used.

The function returns a list of S3 class `gas_bootstrap` with three components: `data`, `model`, `bootstrap`. The `data` component contains the supplied time series and exogenous variables. The `model` component contains the specification of the model structure and size. The `bootstrap` component contains the full set of bootstrapped coefficients as well as the basic statistics derived from them. Available generic functions are `summary()`, `plot()`, `coef()`, and `vcov()`.

3.5 Filtered Parameters

The filtered time-varying parameters of an estimated model can be directly obtained from the output of the `gas()` function. However, to investigate the uncertainty associated with these parameters, the `gas_filter()` function can be used. This function also supports forecasting and provides two methods. The `simulated_coefs` method calculates a path of time-varying parameters for each simulated coefficient set, assuming asymptotic normality with a given variance-covariance matrix. See [Blasques et al. \(2016b\)](#) for more details on this method. The `given_coefs` methods computes a path of time-varying parameters for each supplied coefficient set. Suitable sets of coefficients can be obtained, for example, through the use of the `gas_bootstrap()` function.

An estimated GAS model can be supplied as a `gas` object to the `gas_object` argument:

```
R> gas_filter(gas_object, method = "simulated_coefs", coef_set = NULL,
+   rep_gen = 1000L, t_ahead = 0L, x_ahead = NULL, rep_ahead = 1000L,
+   quant = c(0.025, 0.975))
```

Alternatively, the individual arguments including the data, model specification, and estimated coefficients with variance-covariance matrix can be provided:

```
R> gas_filter(method = "simulated_coefs", coef_set = NULL, rep_gen = 1000L,
+   t_ahead = 0L, x_ahead = NULL, rep_ahead = 1000L, quant = c(0.025, 0.975), y,
+   x = NULL, distr, param = NULL, scaling = "unit", regress = "joint", p = 1L,
+   q = 1L, par_static = NULL, par_link = NULL, par_init = NULL,
+   coef_fix_value = NULL, coef_fix_other = NULL, coef_fix_special = NULL,
+   coef_bound_lower = NULL, coef_bound_upper = NULL, coef_est = NULL,
+   coef_vcov = NULL)
```

The `method` argument determines the approach for capturing uncertainty. For the `given_coefs` method, the `coef_set` argument in the form a numeric matrix of coefficient sets in rows must be provided. For the `simulated_coefs` method, the `rep_gen` argument representing the number of generated coefficient sets must be provided. If forecasting is desired, the number of observations to

forecast can be specified using the `t_ahead` argument, values of exogenous variable for the forecasted period can be provided using the `x_ahead` argument, and the number of simulation repetitions in the forecasted sample can be controlled using the `rep_ahead` argument. The desired quantiles can be specified using the `quant` argument.

The function returns a list of S3 class `gas_filter` with three components: `data`, `model`, `filter`. The `data` component contains the supplied time series and exogenous variables. The `model` component contains the specification of the model structure and size. The `filter` component contains in-sample and possibly out-of-sample means, standard deviations, and quantiles of the time-varying parameters and scores. Available generic functions are `summary()` and `plot()`.

3.6 Supplementary Functions for Distributions

The `distr()` function can be utilized to retrieve a list of distributions and their parametrizations supported by the `gas()` function. To narrow down the output and focus on specific distributions, arguments such as `filter_distr`, `filter_param`, `filter_type`, `filter_dim`, `filter_orthog`, and `filter_default` can be specified. The output is in the form of a `data.frame` with columns providing information on the distributions such as the data type, dimension, orthogonality, and default parameterization.

To work with individual distributions, the `gasmodel` package offers several functions. The `distr_density()` function computes the density of a given distribution, the `distr_mean()` function computes the mean of a given distribution, the `distr_var()` function computes the variance of a given distribution, the `distr_score()` function computes the score of a given distribution, the `distr_fisher()` function computes the Fisher information of a given distribution, and the `distr_random()` function generates random observations from a given distribution. Each of these function can be supplied with arguments specifying the distribution and the parametrization, namely `distr`, `param`, `par_link`. It is important to note that while the `gas()` function may automatically set the logarithmic/logistic link for time-varying parameters, it must be set manually for the distribution functions. Additionally, a vector of parameter values must be provided to the `f` argument. Some functions may also require an observation to be provided to the `y` argument. For detailed usage instructions, please refer to the documentation for each individual function.

4 Case Studies

4.1 Bookshop Orders

In the first case study, we demonstrate the estimation of a univariate GAS model, complemented by bootstrapping and simulation techniques.

We loosely follow [Tomanová and Holý \(2021\)](#) and analyze the timing of orders from a Czech antiquarian bookshop. Besides seasonality and diurnal patterns, one would expect the times of orders to be independent of each other. However, this is not the case and we use a GAS model to capture dependence between the times of orders.

A strand of financial econometrics is devoted to analyzing the timing of transactions by the so-called autoregressive conditional duration (ACD) model introduced by [Engle and Russell \(1998\)](#). For a textbook treatment of such financial point processes, see e.g., [Hautsch and Huang \(2012\)](#).

Let us prepare the analyzed data. We use the `bookshop_sales` dataset containing times of orders from June 8, 2018 to December 20, 2018. We calculate differences of subsequent times, i.e., durations. To avoid zero durations, we set them to 0.5 second.

```
R> library("dplyr")
R> library("tidyr")
R> library("ggplot2")
R> library("hms")
R> library("gasmodel")

R> data("bookshop_sales")
```

```
R> data_dur <- bookshop_sales %>%
+   as_tibble() %>%
+   rename(datetime = time) %>%
+   mutate(date = as.Date(datetime)) %>%
+   mutate(time = as_hms(datetime)) %>%
+   mutate(duration = as.numeric(datetime - lag(datetime)) / 60) %>%
+   mutate(duration = recode(duration, "0" = 0.5)) %>%
+   drop_na()
```

We adjust the observed durations for diurnal pattern and extract the time series to be analyzed.

```
R> model_spl <- smooth.spline(as.vector(data_dur$time), data_dur$duration, df = 10)

R> data_dur <- data_dur %>%
+   mutate(duration_spl = predict(model_spl, x = as.vector(time))$y) %>%
+   mutate(duration_adj = duration / duration_spl)

R> y <- data_dur$duration_adj
```

The following distributions are available for our data type. We utilize the generalized gamma family.

```
R> distr(filter_type = "duration", filter_dim = "uni")
```

	distr_title	param_title	distr	param	type	dim	orthog	default
6	Birnbaum-Saunders	Scale	bisa	scale	duration	uni	TRUE	TRUE
10	Exponential	Rate	exp	rate	duration	uni	TRUE	FALSE
11	Exponential	Scale	exp	scale	duration	uni	TRUE	TRUE
12	Gamma	Rate	gamma	rate	duration	uni	FALSE	FALSE
13	Gamma	Scale	gamma	scale	duration	uni	FALSE	TRUE
14	Generalized Gamma	Rate	gengamma	rate	duration	uni	FALSE	FALSE
15	Generalized Gamma	Scale	gengamma	scale	duration	uni	FALSE	TRUE
31	Weibull	Rate	weibull	rate	duration	uni	FALSE	FALSE
32	Weibull	Scale	weibull	scale	duration	uni	FALSE	TRUE

First, we estimate the model based on the exponential distribution. By default, the logarithmic link for the time-varying scale parameter is adopted. In this particular case, the Fisher information is constant and the three scalings are therefore equivalent.

```
R> est_exp <- gas(y = y, distr = "exp")
R> est_exp
```

GAS Model: Exponential Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	-0.00085202	0.00114896	-0.7416	0.4584
log(scale)_alpha1	0.04888439	0.00650562	7.5142	5.727e-14 ***
log(scale)_phi1	0.96343265	0.00910508	105.8126	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5608.518, AIC: 11223.04, BIC: 11243.01

Second, we estimate the model based on the Weibull distribution. Compared to the exponential distribution, it has an additional shape parameter. By default, the first parameter is assumed time-varying while the remaining are assumed static. In our case, the model features the time-varying scale parameter with the constant shape parameter. However, it is possible to modify this behavior using the `par_static` argument.

```
R> est_weibull <- gas(y = y, distr = "weibull")
R> est_weibull
```

GAS Model: Weibull Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	-0.0019175	0.0013552	-1.4149	0.1571
log(scale)_alpha1	0.0562619	0.0082010	6.8604	6.867e-12 ***
log(scale)_phi1	0.9622643	0.0102230	94.1278	< 2.2e-16 ***
shape	0.9442209	0.0094299	100.1300	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5591.442, AIC: 11190.88, BIC: 11217.51

Third, we estimate the model based on the gamma distribution. This is another generalization of the exponential distribution with an additional shape parameter.

```
R> est_gamma <- gas(y = y, distr = "gamma")
```

```
R> est_gamma
```

GAS Model: Gamma Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	0.0013296	0.0013395	0.9926	0.3209
log(scale)_alpha1	0.0518896	0.0071672	7.2399	4.491e-13 ***
log(scale)_phi1	0.9634327	0.0093853	102.6532	< 2.2e-16 ***
shape	0.9420850	0.0153854	61.2325	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5601.735, AIC: 11211.47, BIC: 11238.1

Fourth, we estimate the model based on the generalized gamma distribution. The generalized gamma distribution encompasses all three aforementioned distributions as special cases.

```
R> est_gengamma <- gas(y = y, distr = "gengamma")
```

```
R> est_gengamma
```

GAS Model: Generalized Gamma Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	-0.049164	0.018903	-2.6009	0.009299 **
log(scale)_alpha1	0.069834	0.011670	5.9841	2.176e-09 ***
log(scale)_phi1	0.951761	0.015024	63.3493	< 2.2e-16 ***
shape1	1.764362	0.150759	11.7032	< 2.2e-16 ***
shape2	0.682971	0.033690	20.2723	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5562.092, AIC: 11134.18, BIC: 11167.47

By comparing the Akaike information criterion (AIC), we find that the most general model, i.e., the one based on the generalized gamma distribution, is the most suitable. For this purpose, we use generic function `AIC()`. Alternatively, the AIC of an estimated model is stored in `est_gengamma$fit$aic`.

```
R> AIC(est_exp, est_weibull, est_gamma, est_gengamma)
```

	df	AIC
est_exp	3	11223.04
est_weibull	4	11190.88
est_gamma	4	11211.47
est_gengamma	5	11134.18

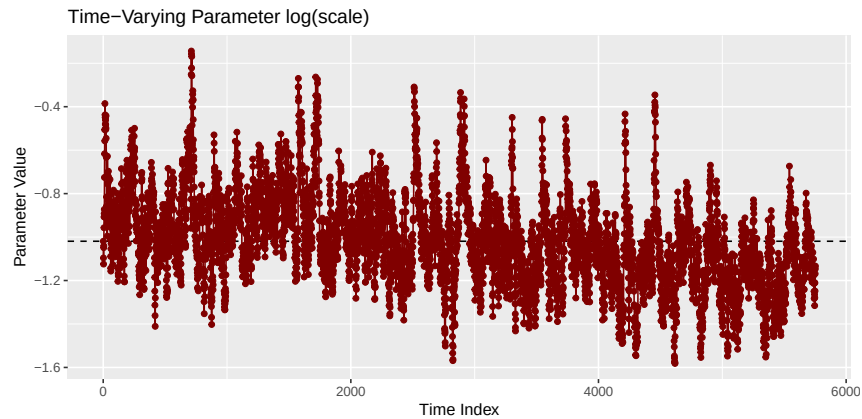


Figure 3: Time-varying parameters based on the generalized gamma model.

Let us take a look on the time-varying parameters of the generalized gamma model (Figure 3).

```
R> plot(est_gengamma)
```

We can see a slight negative trend in time-varying parameters. We can try including a trend as an exogenous variable for all four considered distributions.

```
R> x <- as.integer(data_dur$date) - as.integer(data_dur$date[1])
```

```
R> est_exp_tr <- gas(y = y, x = x, distr = "exp", reg = "sep")
```

```
R> est_exp_tr
```

GAS Model: Exponential Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	0.29683416	0.04509203	6.5829	4.615e-11 ***
log(scale)_beta1	-0.00304957	0.00037137	-8.2118	< 2.2e-16 ***
log(scale)_alpha1	0.05401728	0.00802442	6.7316	1.678e-11 ***
log(scale)_phi1	0.91358230	0.02146703	42.5575	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5583.723, AIC: 11175.45, BIC: 11202.08

```
R> est_weibull_tr <- gas(y = y, x = x, distr = "weibull", reg = "sep")
```

```
R> est_weibull_tr
```

GAS Model: Weibull Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	0.26955739	0.04763575	5.6587	1.525e-08 ***
log(scale)_beta1	-0.00302892	0.00039014	-7.7638	8.244e-15 ***
log(scale)_alpha1	0.06215424	0.00992563	6.2620	3.801e-10 ***
log(scale)_phi1	0.90950196	0.02399584	37.9025	< 2.2e-16 ***
shape	0.94858384	0.00949927	99.8586	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5569.405, AIC: 11148.81, BIC: 11182.1

```
R> est_gamma_tr <- gas(y = y, x = x, distr = "gamma", reg = "sep")
```

```
R> est_gamma_tr
```


GAS Model: Gamma Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	0.35024097	0.04910603	7.1323	9.868e-13 ***
log(scale)_beta1	-0.00304957	0.00038142	-7.9954	1.292e-15 ***
log(scale)_alpha1	0.05698059	0.00874363	6.5168	7.182e-11 ***
log(scale)_phi1	0.91358230	0.02204841	41.4353	< 2.2e-16 ***
shape	0.94799429	0.01549052	61.1983	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5578.303, AIC: 11166.61, BIC: 11199.89

```
R> est_gengamma_tr <- gas(y = y, x = x, distr = "gengamma", reg = "sep")
R> est_gengamma_tr
```

GAS Model: Generalized Gamma Distribution / Scale Parametrization / Unit Scaling

Coefficients:

	Estimate	Std. Error	Z-Test	Pr(> Z)
log(scale)_omega	-0.70489163	0.19283438	-3.6554	0.0002568 ***
log(scale)_beta1	-0.00292746	0.00039123	-7.4827	7.280e-14 ***
log(scale)_alpha1	0.08164957	0.01387329	5.8854	3.971e-09 ***
log(scale)_phi1	0.87684184	0.03506612	25.0054	< 2.2e-16 ***
shape1	1.76342697	0.15253550	11.5608	< 2.2e-16 ***
shape2	0.68568220	0.03426457	20.0114	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Log-Likelihood: -5541.097, AIC: 11094.19, BIC: 11134.14

The trend variable is significant in all cases. The AIC also confirms improvement of the fit.

```
R> AIC(est_exp_tr, est_weibull_tr, est_gamma_tr, est_gengamma_tr)
```

	df	AIC
est_exp_tr	4	11175.45
est_weibull_tr	5	11148.81
est_gamma_tr	5	11166.61
est_gengamma_tr	6	11094.19

Note that the time-varying parameters returned by the `gas()` function include the effect of exogenous variables. By using the `plot()` function, the now modeled trend can be clearly seen (Figure 4).

```
R> plot(est_gengamma_tr)
```

To assess the suitability of standard deviations based on asymptotics for our finite sample, we employ the `gas_bootstrap()` function. This function conducts a parametric bootstrap, allowing us to calculate standard errors and quantiles. It's important to note that this could be computationally very intensive, depending on the number of repetitions, the quantity of observations, the complexity of the model structure, and the optimizer used. The function supports parallelization through arguments `parallel_function` and `parallel_arguments`. For example, for the `snow` parallelization functionality with 4 cores, you can call `gas_bootstrap(est_gengamma_tr, parallel_function = wrapper_parallel_snow, parallel_arguments = list(spec = 4))`.

```
R> set.seed(42)
R> boot_gengamma_tr <- gas_bootstrap(est_gengamma_tr, method = "parametric")
R> boot_gengamma_tr
```

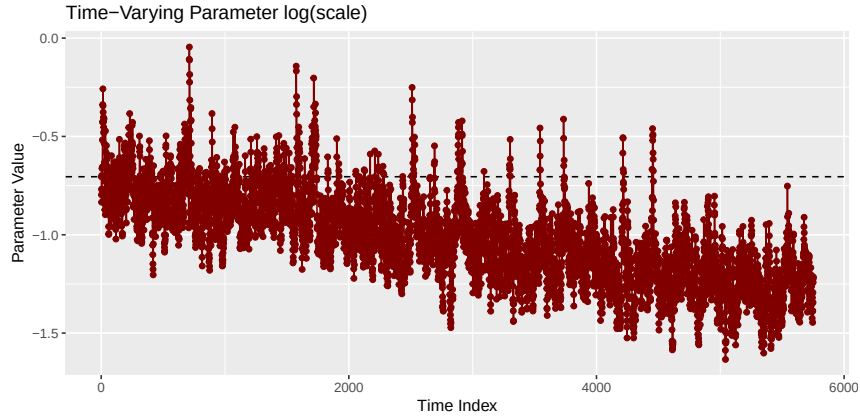


Figure 4: Time-varying parameters based on the generalized gamma model with trend.

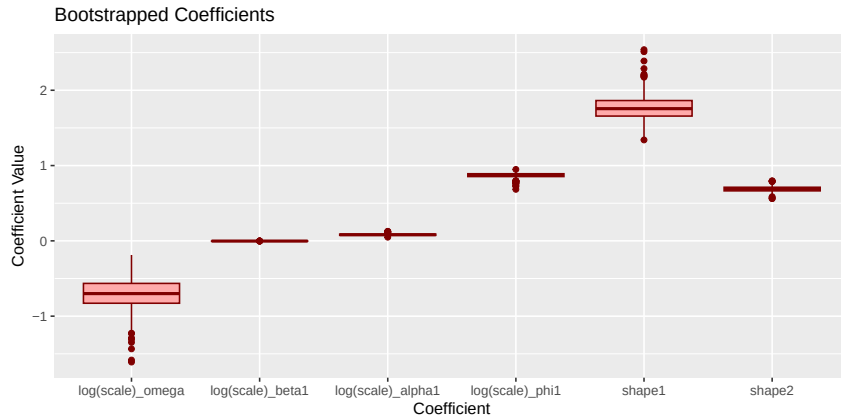


Figure 5: Boxplot of bootstrapped coefficients based on the generalized gamma model with trend.

GAS Model: Generalized Gamma Distribution / Scale Parametrization / Unit Scaling

Method: Parametric Bootstrap

Number of Bootstrap Samples: 1000

Bootstrapped Coefficients:

	Original	Mean	Std. Error	P-Value	2.5%	97.5%
log(scale)_omega	-0.704891626	-0.705226357	0.1980063054	0	-1.110703445	-0.34772409
log(scale)_beta1	-0.002927462	-0.002932047	0.0003883289	0	-0.003698968	-0.00217132
log(scale)_alpha1	0.081649573	0.081648774	0.0116918758	0	0.059179321	0.10575927
log(scale)_phi1	0.876841843	0.871223702	0.0297172317	0	0.806282230	0.91973863
shape1	1.763426971	1.765458333	0.1596470324	0	1.476831790	2.09983786
shape2	0.685682201	0.688266197	0.0354034049	0	0.623633255	0.76097862

The results can also be viewed in a boxplot (Figure 5).

```
R> plot(boot_gengamma_tr)
```

Given that the number of observations in our model is 5752 (accessible through `est_gengamma_tr$model$t`), it is reasonable to anticipate that standard deviations based on asymptotics would yield precise results. Fortunately, this holds true in our scenario. Note that standard deviations can also be obtained using the `vcov()` generic function for both `est_gengamma_tr` and `boot_gengamma_tr`.

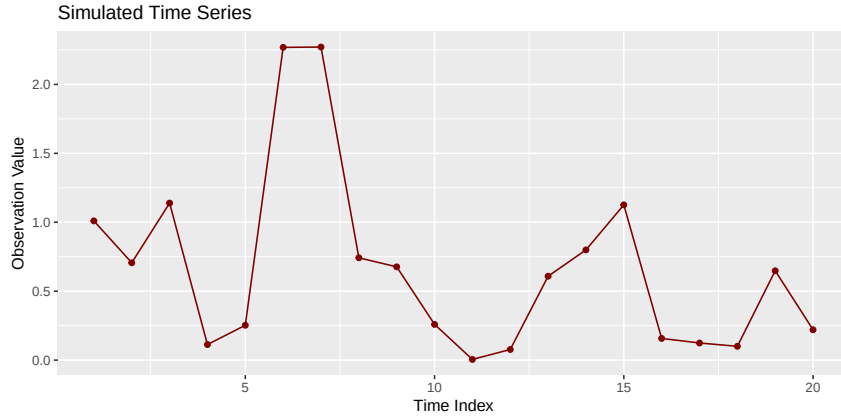


Figure 6: Simulated time series based on the generalized gamma model with trend.

```
R> est_gengamma_tr$fit$coef_sd - boot_gengamma_tr$bootstrap$coef_sd

log(scale)_omega log(scale)_beta1 log(scale)_alpha1 log(scale)_phi1 shape1
-5.171928e-03    2.900388e-06    2.181417e-03    5.348887e-03    -7.111530e-03
shape2
-1.138839e-03
```

Lastly, we highlight the utilization of simulation techniques. Simulation is executed using the `gas_simulate()` function, which can be supplied with either an estimated model or a custom model structure.

```
R> t_sim <- 20
R> x_sim <- rep(max(x) + 1, t_sim)

R> set.seed(42)
R> sim_gengamma_tr <- gas_simulate(est_gengamma_tr, t_sim = t_sim, x_sim = x_sim)
R> sim_gengamma_tr
```

GAS Model: Generalized Gamma Distribution / Scale Parametrization / Unit Scaling

```
Simulations:
      t1      t2      t3      t4      t5      t6      t7
1.009836881 0.706070572 1.139254609 0.112834862 0.252712188 2.268641670 2.271065825
      t8      t9      t10     t11     t12     t13     t14
0.742231695 0.676595922 0.259042333 0.004836128 0.077080566 0.608510890 0.799449725
      t15     t16     t17     t18     t19     t20
1.126124047 0.157351783 0.124067217 0.100168697 0.648121920 0.219983546
```

The simulated time series can be plotted using the generic `plot()` function (Figure 6).

```
R> plot(sim_gengamma_tr)
```

The simulated time series can be employed, for example, to assess the impact of order arrivals on queuing systems, as demonstrated by [Tomanová and Holý \(2021\)](#).

4.2 Ice Hockey Rankings

In the second case study, we showcase the estimation of a multivariate GAS model, followed by forecasting and assessing uncertainty in the filtered time-varying parameters.

We present the empirical study of [Holý and Zouhar \(2022\)](#) which analyzes the results of the Ice Hockey World Championships. Our main object of interest is the annual ranking of 16 teams participating in the championships. While there exists a comprehensive statistical toolkit for ranking

data, as described e.g., by [Alvo and Yu \(2014\)](#), it is worth noting that the time perspective is often overlooked in the ranking literature, as highlighted by [Yu *et al.* \(2019\)](#). This is precisely where the GAS model emerges as a valuable tool in our analysis.

Our analyzed data are supplied in the `ice_hockey_championships` dataset. We restrict ourselves to years 1998–2019 just as [Holý and Zouhar \(2022\)](#). In 1998, the number of teams in the tournament increased from 12 to 16. In 2020, the championship was cancelled due to Covid-19 pandemic. We start by creating two variables – the final ranking of 16 participating teams in each year `y` and the dummy variable indicating which country (or countries) hosted the championship in each year `x`.

```
R> library("dplyr")
R> library("ggplot2")
R> library("gasmodel")

R> data("ice_hockey_championships")

R> t <- 22
R> n <- ncol(ice_hockey_championships$host)
R> y <- ice_hockey_championships$rankings[1:t, ]
R> x <- setNames(lapply(1:n, function(i) { ice_hockey_championships$host[1:t, i] } ),
+   colnames(y))
```

We look at some basic statistics. In our sample, nine countries have participated each year.

```
R> participate <- colSums(is.finite(y))
R> names(participate)[participate == t]

[1] "CAN" "CHE" "CZE" "FIN" "LVA" "RUS" "SVK" "SWE" "USA"
```

The following countries hosted the championships, some of them multiple times.

```
R> host <- sapply(x, FUN = sum)
R> host[host > 0L]

AUT BLR CAN CHE CZE DEU FIN FRA LVA NOR RUS SVK SWE
  1   1   1   2   2   3   3   1   1   1   3   2   3
```

In the years under analysis, the gold medals were awarded to the following countries.

```
R> gold <- colSums(y == 1L)
R> gold[gold > 0L]

CAN CZE FIN RUS SVK SWE
  5   5   2   4   1   5
```

The **gasmodel** package provides a single distribution on rankings – the Plackett–Luce distribution.

```
R> distr(filter_type = "ranking")

      distr_title param_title distr param      type    dim orthog default
24 Plackett-Luce      Worth pluce worth ranking multi  FALSE      TRUE
```

It is a convenient and simple probability distribution on rankings utilizing a worth parameter for each item to be ranked. It originates from Luce’s choice axiom and is also related to the Thurstone’s theory of comparative judgment, see [Luce \(1977\)](#) and [Yellott \(1977\)](#). For more details on this distribution, see [Plackett \(1975\)](#), [Stern \(1990\)](#), and [Critchlow *et al.* \(1991\)](#).

We consider a total of three different models. We incorporate `x` as an exogenous variable in our model to capture possible home advantage. For each model, we assume a panel-like structure where each worth parameter has its own intercept, while the regression and dynamics parameters remain the same for all worth parameters. In the **gasmodel** package, this structure can be achieved

using the `coef_fix_value` and `coef_fix_other` arguments. Alternatively, for convenience, the value `panel_structure` can be included in the `coef_fix_special` argument. It is important to note that the worth parameters in the Plackett–Luce distribution are not identifiable, and it is common practice to impose a standardizing condition. In our model, we enforce the condition that the sum of all ω_i is 0. This can be accomplished by including the value `zero_sum_intercept` in the `coef_fix_special` argument.

First, we estimate the static model where there are no dynamics involved. In this case, we set both the autoregressive and score orders to zero. Either a single integer can be provided to determine the order for all parameters, or a vector of integers can be supplied to specify the order for individual parameters.

```
R> est_static <- gas(y = y, x = x, distr = "pluce", p = 0, q = 0,
+   coef_fix_special = c("zero_sum_intercept", "panel_structure"))
```

Second, we estimate the standard mean-reverting GAS model of order one. In order to expedite the numerical optimization process, we incorporate starting values based on the static model.

```
R> est_stnry <- gas(y = y, x = x, distr = "pluce",
+   coef_fix_special = c("zero_sum_intercept", "panel_structure"),
+   coef_start = as.vector(rbind(est_static$fit$par_unc / 2, 0, 0.5, 0.5)))
```

Third, we estimate the random walk model. In other words, we set the autoregressive coefficient to 1. The easiest way to specify this is by including the value `random_walk` in the `coef_fix_special` argument. In our random walk model, we consider the initial values of the worth parameters to be parameters to be estimated. While the `par_init` argument does not directly support this, we can set `regress = "sep"` and use cumulative sums of exogenous variables to achieve this initialization for this particular model. However, it is generally not recommended to estimate initial parameter values as it introduces additional variables, lacks reasonable asymptotics, and can lead to overfitting in finite samples. It is important to approach the random walk model with caution, as it is not stationary and the standard maximum likelihood asymptotics are not valid.

```
R> est_walk <- gas(y = y, x = lapply(x, cumsum), distr = "pluce", regress = "sep",
+   coef_fix_special = c("zero_sum_intercept", "panel_structure", "random_walk"),
+   coef_start = as.vector(rbind(est_static$fit$par_unc, 0, 0.5, 1)))
```

To avoid redundancy, we will omit the output of the `gas()` function, which contains rows for each coefficient of each worth parameter. Since most coefficients are the same due to the assumed panel structure, it is unnecessary to display them all. Instead, we print only one set of the home advantage and dynamics coefficients.

```
R> cbind(est_static = c("beta1" = unname(coef(est_static)[2]), "alpha1" = 0, "phi1" = 0),
+   est_stnry = coef(est_stnry)[2:4], est_walk = coef(est_walk)[2:4])
```

	est_static	est_stnry	est_walk
beta1	0.1707329	0.2274378	0.09873335
alpha1	0.0000000	0.3919432	0.34300137
phi1	0.0000000	0.5062479	1.00000000

In all three models, coefficient β_1 representing the home advantage is positive but not significant.

```
R> cbind(est_static = c("beta1" = unname(est_static$fit$coef_pval)[2], "alpha1" = 0, "phi1" = 0),
+   est_stnry = est_stnry$fit$coef_pval[2:4], est_walk = est_walk$fit$coef_pval[2:4])
```

	est_static	est_stnry	est_walk
beta1	0.514887	3.772815e-01	5.995825e-01
alpha1	0.000000	2.141361e-06	2.634611e-09
phi1	0.000000	6.463353e-04	0.000000e+00

We compare the models using the Akaike information criterion (AIC). The `gas` class allows for generic function `AIC()`. In terms of AIC, the mean-reverting model outperformed the remaining two by a wide margin.

```
R> AIC(est_static, est_stnry, est_walk)
```

```

      df      AIC
est_static 24 1299.600
est_stnry  26 1274.391
est_walk   25 1300.851

```

Our models enable us to construct the “ultimate” or long-run ranking. The rankings produced by both models are in agreement for all but the first three positions. However, the long-term strength estimates for these three teams are very close to each other, making the final ranking less clear-cut.

```

R> tibble(team = colnames(y)) %>%
+   mutate(stnry_strength = est_stnry$fit$par_unc) %>%
+   mutate(stnry_rank = rank(-stnry_strength)) %>%
+   mutate(static_strength = est_static$fit$par_unc) %>%
+   mutate(static_rank = rank(-static_strength)) %>%
+   arrange(stnry_rank)

```

```
# A tibble: 24 × 5
```

	team	stnry_strength	stnry_rank	static_strength	static_rank
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	CAN	3.72	1	3.72	2
2	FIN	3.70	2	3.66	3
3	SWE	3.65	3	3.84	1
4	CZE	3.47	4	3.41	4
5	RUS	3.25	5	3.17	5
6	USA	1.83	6	2.18	6
7	CHE	1.67	7	1.76	7
8	SVK	1.65	8	1.55	8
9	LVA	0.862	9	0.822	9
10	DEU	0.280	10	0.311	10
11	BLR	0.254	11	0.109	11
12	NOR	0.0335	12	-0.0743	12
13	DNK	-0.0732	13	-0.175	13
14	FRA	-0.405	14	-0.509	14
15	AUT	-0.833	15	-0.886	15
16	ITA	-1.02	16	-1.10	16
17	UKR	-1.34	17	-1.52	17
18	SVN	-1.75	18	-1.64	18
19	KAZ	-1.83	19	-1.78	19
20	JPN	-1.99	20	-1.94	20
21	HUN	-3.28	21	-3.20	21
22	GBR	-3.92	22	-3.89	22
23	POL	-3.95	23	-3.90	23
24	KOR	-3.96	24	-3.91	24

Additionally, we can examine the evolution of the worth parameters for individual teams over the years. The point estimates of time-varying parameter values can be directly obtained from the `gas()` function. Using the generic `plot()` function allows us to visualize the time-varying parameters of individual models. When multiple parameters are time-varying, as in our scenario, the function plots them in sequence. For the purpose of this document, we will only display figures specific to the Canada team (Figures 7, 8, and 9).

```

R> plot(est_static, which = 3)
R> plot(est_stnry, which = 3)
R> plot(est_walk, which = 3)

```

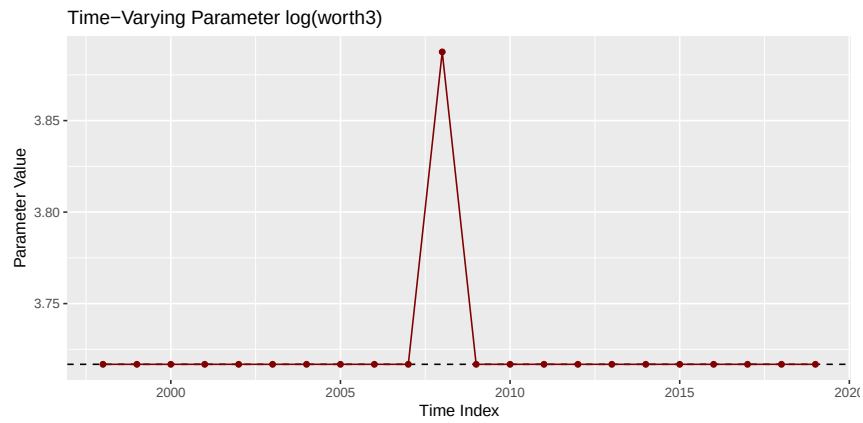


Figure 7: Time-varying parameters of the Canada team based on the static model.

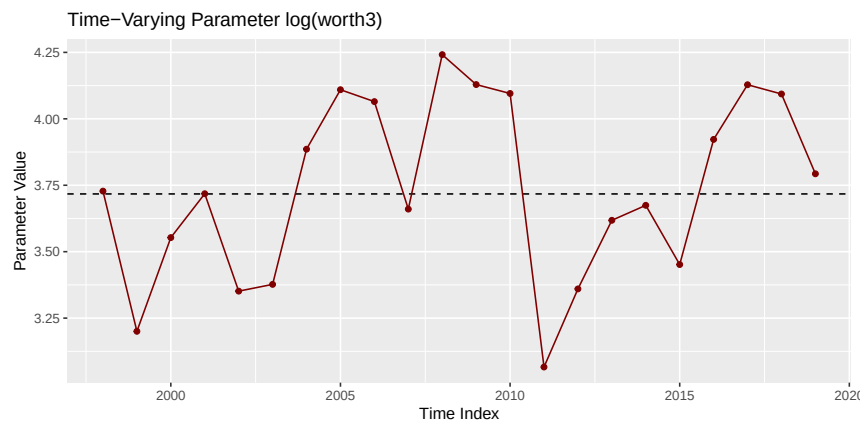


Figure 8: Time-varying parameters of the Canada team based on the stationary model.



Figure 9: Time-varying parameters of the Canada team based on the random walk model.

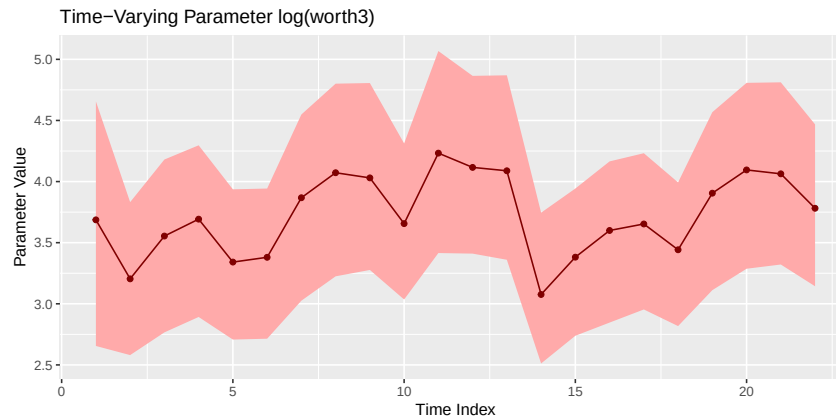


Figure 10: Confidence bands of time-varying parameters of the Canada team based on the stationary model.

However, it is important to note that these estimates are subject to uncertainty. To capture the uncertainty, we can utilize simulations by leveraging the `gas_filter()` function, which accepts the output of the `gas()` function as an argument. This allows us to obtain the standard deviations and quantiles for the worth parameter estimates, providing a more comprehensive understanding of the parameter dynamics over time.

```
R> set.seed(42)
R> flt_stnry <- gas_filter(est_stnry)
```

To visualize time-varying parameters with confidence band, we can use the `plot()` on the `gas_filter` object (Figure 10).

```
R> plot(flt_stnry, which = 3)
```

Finally, we perform one-year-ahead forecasts. We use the `gas_forecast()` function, which can again take the estimated model as an argument.

```
R> fcst_stnry <- gas_forecast(est_stnry, t_ahead = 1, x_ahead = 0)

R> tibble(team = colnames(y)) %>%
+   mutate(fcst_strength = fcst_stnry$forecast$par_tv_ahead_mean[1, ]) %>%
+   mutate(fcst_gold = exp(fcst_strength) / sum(exp(fcst_strength))) %>%
+   mutate(fcst_rank = rank(-fcst_strength)) %>%
+   mutate(real_rank = ice_hockey_championships$rankings[24, ]) %>%
+   arrange(real_rank)

# A tibble: 24 × 5
  team fcst_strength fcst_gold fcst_rank real_rank
<chr> <dbl> <dbl> <dbl> <dbl>
1 CAN 3.97 0.234 2 1
2 FIN 3.97 0.235 1 2
3 USA 2.09 0.0356 6 3
4 DEU 0.742 0.00929 10 4
5 RUS 3.43 0.137 3 5
6 CHE 1.82 0.0272 7 6
7 CZE 3.41 0.134 4 7
8 SVK 1.58 0.0214 8 8
9 SWE 3.40 0.133 5 9
10 KAZ -2.05 0.000569 19 10
11 LVA 0.978 0.0117 9 11
12 DNK 0.229 0.00556 11 12
13 NOR 0.125 0.00501 12 13
```

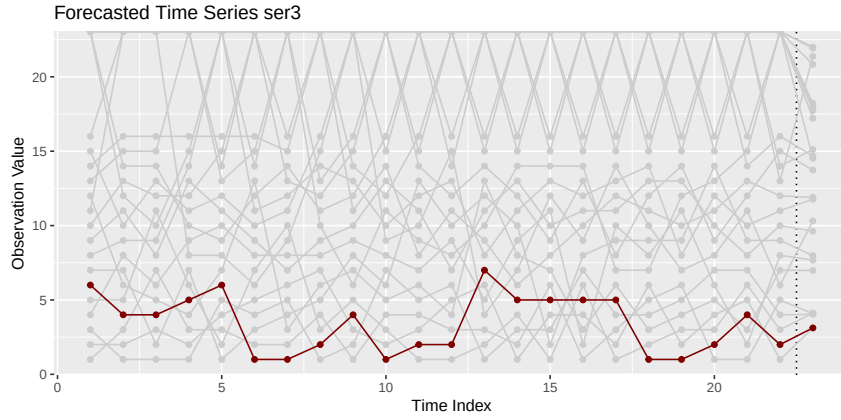


Figure 11: One-step ahead forecasts of the Canada team based on the stationary model.

14	GBR	-3.55	0.000127	22	14
15	BLR	-0.704	0.00219	14	15
16	ITA	-0.957	0.00170	16	16
17	AUT	-0.832	0.00192	15	Inf
18	FRA	-0.490	0.00271	13	Inf
19	HUN	-3.31	0.000162	21	Inf
20	JPN	-2.21	0.000486	20	Inf
21	KOR	-3.81	0.0000982	23	Inf
22	POL	-3.99	0.0000821	24	Inf
23	SVN	-1.95	0.000631	18	Inf
24	UKR	-1.69	0.000813	17	Inf

The forecasted values can be displayed using the generic `plot()` function (Figure 11).

```
R> plot(fcst_stnry, which = 3)
```

5 Limitations and Customization

5.1 Adding a New Distribution

Despite providing a reasonable range of distributions (refer to Table 1), the current version of the **gasmodel** package does not include certain distributions found in the GAS literature. Notable examples are copula models (see, e.g., [De Lira Salvatierra and Patton, 2015](#); [Koopman *et al.*, 2018](#)), matrix models (see, e.g., [Hansen *et al.*, 2016](#); [Opschoor *et al.*, 2018](#)), and censoring models (see, e.g., [Harvey and Ito, 2020](#); [Harvey and Liao, 2023](#)).

Users are encouraged to customize the package by adding new distributions. To incorporate a new distribution into the package, please follow these steps:

1. Choose a name for the distribution and parametrization, such as `newdistr` and `newparam`, respectively.
2. Create an R file in the R directory, such as `R/distr_newdistr_newparam.R`, which will contain all the necessary functions for the new distribution.
3. Implement the following functions in the R file, adhering to the structure used for other distributions in the package:
 - `distr_newdistr_newparam_parameters()` listing the parameters,
 - `distr_newdistr_newparam_density()` computing the density,

- `distr_newdistr_newparam_loglik()` computing the log-likelihood,
 - `distr_newdistr_newparam_mean()` computing the mean,
 - `distr_newdistr_newparam_var()` computing the variance,
 - `distr_newdistr_newparam_score()` computing the score,
 - `distr_newdistr_newparam_fisher()` computing the Fisher information,
 - `distr_newdistr_newparam_random()` generating random variables,
 - `distr_newdistr_newparam_start()` estimating starting values of the parameters.
4. Update the `distr_table.xlsx` file located in the `data-raw` directory by adding a new row to the table that includes the names of the distribution and parametrization.
 5. Run the `distr_table.R` script located in the `data-raw` directory. This script saves the content of the `distr_table.xlsx` table to the `distr_table` dataset in the package.

By following these steps, users will be able to add a novel distribution to the package, integrating it with the existing framework.

5.2 Interaction Between Parameters and Non-Linear Dependence

The dynamics are implemented in the standard form (3) of [Creal *et al.* \(2013\)](#), which has been further extended to include exogenous variables in the form (4) or (5). However, it is worth noting that existing literature includes models with interactions between different time-varying parameters or nonlinear forms of dependence on past values (see, e.g., [Harvey and Sucarrat, 2014](#); [Holý and Tomanová, 2022](#)). Incorporating such complex dynamics would significantly complicate the interface of the functions, so we have opted to keep the dynamics simple for ease of use.

Nevertheless, the source code can be modified to accommodate specific cases. This can be achieved by using a placeholder exogeneous variable and making a manual adjustment within the `likelihood_evaluate()` function in the `helper_likelihood.R` file. Specifically, the value of the placeholder variable can be hard-coded to a desirable transformation of any concurrent or lagged parameter.

5.3 Non-Standard Structure of Time Series

The package focuses on the standard form of time series. However, certain applications, such as those in the field of sports statistics, may require a specialized structure for modeling time series data. In these cases, the individual matches between teams or players in a specific league are often modeled using distributions like Bernoulli, Skellam, or bivariate Poisson (see, e.g., [Gorgi *et al.*, 2019](#); [Koopman and Lit, 2019](#)). Time series should therefore represent the outcomes of matches. However, at each observation, different teams may be participating. This unique characteristic cannot be adequately captured by the standard form of univariate (or bivariate) time series, and a more sophisticated data structure is required to account for the varying teams involved.

To address this limitation, an R package that specifically caters to the use of score-driven models in sports statistics is currently being developed. This specialized package will provide the necessary tools and data structures to effectively model and analyze the unique dynamics present in these applications. However, there are other options beyond R that already exist. Notably, GAS pairwise comparison models can be estimated using the **PyFlux** package in **Python** ([Taylor, 2018](#)), as well as through the stand-alone GUI application **Time Series Lab** ([Lit *et al.*, 2021](#)).

5.4 Other Dynamic Models Using Score

In the literature on GAS models, the score has been employed in a wide range of dynamic models. Some of these models fall outside the scope of this package. Examples of such models include semi-parametric models (see, e.g., [Blasques *et al.*, 2016a](#); [Patton *et al.*, 2019](#)), Markov regime switching models (see, e.g., [Bazzi *et al.*, 2017](#); [Blazsek and Haddad, 2022](#)), and spatio-temporal models (see, e.g., [Catania and Billé, 2017](#); [Gasperoni *et al.*, 2023](#)).

6 Conclusion

The purpose of the **gasmodel** package is to provide researchers, analysts, and data scientists with a versatile toolkit for a broad spectrum of GAS models in R. While it is important to note that not all GAS models found in the literature are supported by the package due to their diverse nature, the package still provides a solid foundation. For some specific GAS models, modifications of the package may be required, or an alternative specialized package/code may prove to be a better option. Nevertheless, the **gasmodel** package offers considerable flexibility for specifying dynamics, and it boasts an extensive array of probability distribution options. This ensures that users have a diverse set of tools at their disposal when working with GAS models, enabling them to tailor their analyses to their specific needs.

Funding

The work on this paper was supported by the Czech Science Foundation under project 23-06139S and the personal and professional development support program of the Faculty of Informatics and Statistics, Prague University of Economics and Business.

References

- Alvo M, Yu PLH (2014). *Statistical Methods for Ranking Data*. Springer, New York. ISBN 978-1-4939-1470-8. <https://doi.org/10.1007/978-1-4939-1471-5>.
- Ardia D, Boudt K, Catania L (2019). “Generalized Autoregressive Score Models in R: The GAS Package.” *Journal of Statistical Software*, **88**(6), 1–28. ISSN 1548-7660. <https://doi.org/10.18637/jss.v088.i06>.
- Artemova M, Blasques F, van Brummelen J, Koopman SJ (2022). “Score-Driven Models: Methods and Applications.” *Oxford Research Encyclopedia of Economics and Finance*. <https://doi.org/10.1093/acrefore/9780190625979.013.671>.
- Bazzi M, Blasques F, Koopman SJ, Lucas A (2017). “Time-Varying Transition Probabilities for Markov Regime Switching Models.” *Journal of Time Series Analysis*, **38**(3), 458–478. ISSN 0143-9782. <https://doi.org/10.1111/jtsa.12211>.
- Blasques F, Gorgi P, Koopman SJ, Wintenberger O (2018). “Feasible Invertibility Conditions and Maximum Likelihood Estimation for Observation-Driven Models.” *Electronic Journal of Statistics*, **12**(1), 1019–1052. ISSN 1935-7524. <https://doi.org/10.1214/18-ejs1416>.
- Blasques F, Holý V, Tomanová P (2022a). “Zero-Inflated Autoregressive Conditional Duration Model for Discrete Trade Durations with Excessive Zeros.” <https://arxiv.org/abs/1812.07318>.
- Blasques F, Ji J, Lucas A (2016a). “Semiparametric Score Driven Volatility Models.” *Computational Statistics & Data Analysis*, **100**(2013), 58–69. ISSN 0167-9473. <https://doi.org/10.1016/j.csda.2015.04.003>.
- Blasques F, Koopman SJ, Łasak K, Lucas A (2016b). “In-Sample Confidence Bands and Out-of-Sample Forecast Bands for Time-Varying Parameters in Observation-Driven Models.” *International Journal of Forecasting*, **32**(3), 875–887. ISSN 0169-2070. <https://doi.org/10.1016/j.ijforecast.2015.11.018>.
- Blasques F, Koopman SJ, Lucas A (2014). “Stationarity and Ergodicity of Univariate Generalized Autoregressive Score Processes.” *Electronic Journal of Statistics*, **8**(1), 1088–1112. ISSN 1935-7524. <https://doi.org/10.1214/14-ejs924>.